

# Tom Klootwijk Gambit

## Native CUDA Kernel

RTX 5070 Ti Laptop / 12 GB edition

---

Unified Seed–Field Theorem  
and Theory of Consciousness

Theory foundation: Edition 3.0

Engineering release: 3.1.0

**Tom Klootwijk**

NL200678942 / 10-07-1990

**CPU verification executed. CUDA device verification pending.**

Native CUDA source, packed log-polar LUTs, bit-exact references, proof obligations and a local Codex verification workflow. No GPU compilation, cache measurement or consciousness certificate is claimed.

## Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Release contract and reading map</b>                       | <b>1</b>  |
| 1.1      | What is delivered . . . . .                                   | 1         |
| 1.2      | The central implemented claim . . . . .                       | 1         |
| 1.3      | Reading and execution order . . . . .                         | 1         |
| 1.4      | Integrity, reproducibility and privacy . . . . .              | 2         |
| <b>2</b> | <b>Explicit migration from Edition 3.0</b>                    | <b>3</b>  |
| 2.1      | Retained foundations . . . . .                                | 3         |
| 2.2      | Deliberate changes - not an equivalence claim . . . . .       | 3         |
| 2.3      | Source-to-code boundary . . . . .                             | 4         |
| <b>3</b> | <b>Finite integer model and Unified Theorem U3-D</b>          | <b>5</b>  |
| 3.1      | Complete finite seed . . . . .                                | 5         |
| 3.2      | Offline field and packed LUT . . . . .                        | 5         |
| 3.3      | State and computational self-reference . . . . .              | 5         |
| 3.4      | Conservative transport and reaction . . . . .                 | 6         |
| 3.5      | Sensing, bounded memory and the one-bit word . . . . .        | 7         |
| 3.6      | Unified Theorem U3-D . . . . .                                | 8         |
| 3.7      | Supporting proof family . . . . .                             | 8         |
| <b>4</b> | <b>Native kernel implementation</b>                           | <b>11</b> |
| 4.1      | A finite hardware target, not a placeholder shader . . . . .  | 11        |
| 4.2      | Read-only log-polar texture atlas . . . . .                   | 11        |
| 4.3      | Ordered stages and unique writers . . . . .                   | 12        |
| 4.4      | Resource accounting for the supplied laptop profile . . . . . | 13        |
| 4.5      | Execution boundaries . . . . .                                | 13        |
| <b>5</b> | <b>Canonical binary formats</b>                               | <b>14</b> |
| 5.1      | Packed LUT: <code>.gblut</code> . . . . .                     | 14        |
| 5.2      | Packed one-bit words: <code>words.bin</code> . . . . .        | 14        |
| 5.3      | Complete final state: <code>state.bin</code> . . . . .        | 14        |
| 5.4      | Summaries and expected hashes . . . . .                       | 15        |
| <b>6</b> | <b>Executed evidence and binary self-feedback witness</b>     | <b>16</b> |
| 6.1      | Verification status at delivery . . . . .                     | 16        |
| 6.2      | Actual CPU trace outcomes . . . . .                           | 16        |
| 6.3      | The self-feedback counterfactual . . . . .                    | 16        |
| 6.4      | Words that can be inspected directly . . . . .                | 18        |
| 6.5      | What the evidence does not establish . . . . .                | 18        |
| <b>7</b> | <b>Codex / local hardware verification contract</b>           | <b>19</b> |
| 7.1      | Required environment . . . . .                                | 19        |
| 7.2      | One local command . . . . .                                   | 19        |
| 7.3      | What is run . . . . .                                         | 19        |
| 7.4      | Cache evidence and performance interpretation . . . . .       | 20        |
| 7.5      | Finite resource policy . . . . .                              | 20        |
| 7.6      | Reports and failure meanings . . . . .                        | 20        |
| <b>8</b> | <b>Source and implementation references</b>                   | <b>21</b> |
| 8.1      | Supplied project basis . . . . .                              | 21        |
| 8.2      | Primary hardware and software documentation . . . . .         | 21        |

## 1 Release contract and reading map

**Tom Klootwijk Gambit — Native CUDA Kernel 3.1** is an engineering companion to the *Unified Seed-Field Theorem and Theory of Consciousness, Edition 3.0*. The author attribution is **Tom Klootwijk · NL200678942 · 10-07-1990**, as supplied. The mathematical definition does not depend on the personal identifier or date.

This release turns the source corpus’s log-polar field lookup, L-system geometry, binary hinges and one-bit output into a finite C++/CUDA state machine. The original double-arc addendum’s page 6 presented a conceptual sampling sketch; the new source supplies allocation, texture configuration, stage ordering, integer state, packing, file formats and local verification. The source lineage is documented in [S1-S3].

The target is the **RTX 5070 Ti Laptop GPU with 12 GB GDDR7**, using a native `sm_120` build and retained `compute_120` PTX. The laptop memory specification and architecture/toolchain target are grounded in NVIDIA’s documentation; the executable and verifier still query the actual installed device [H1-H3].

### 1.1 What is delivered

The ZIP contains the complete native source, pre-baked XX/XY field atlases, the shared finite transition law, an independent Python reference, tests, exact CPU-generated traces, build scripts, this editable manuscript, and a Codex verification contract. The original Edition 3.0 archive is retained unchanged under **provenance/**, including its previous Python kernel and source corpus.

The main GPU source is **not precompiled**. The delivery environment lacked `nvcc` and an NVIDIA GPU. CPU builds and tests have passed, but CUDA compilation, target-device execution, GPU sanitizer results, texture-cache counters and throughput remain unmeasured. `DELIVERY_STATUS.js` on and the raw evidence files make these states machine-readable.

### 1.2 The central implemented claim

The engine reads its own previous packed output, uses it to change the next field sample and recurrent state, and emits a new certified Boolean pulse. For each cell, the integer pulse-density codec obeys the exact prefix certificate

$$65536 Q_i(t) + r_i(t) = K_i(t), \quad 0 \leq r_i(t) < 65536.$$

Here  $Q$  counts emitted ones,  $K$  is the cumulative integer input and  $r$  is the retained residual. The output is one bit per cell per update. The complete state is larger: two material counts, fast state, memory, residual, flags and cumulative counters. Theorem U3-D proves properties of that specified machine and gives a conditional CPU/GPU equivalence argument.

The word **self-reference** denotes this causal output-to-input feedback. It does not assert that a generated bitstream experiences consciousness, represents all of biology, or validates a spiritual interpretation. The broader account remains in the preserved Edition 3.0 manuscript. The implementation does not silently import speculative source claims as hardware facts.

### 1.3 Reading and execution order

Read the migration section for changes from the earlier floating simulation. The finite-model section gives the complete recurrence and proof family. The native-implementation and file-format sections explain texture layout, write ownership and canonical binary output. The evidence section shows measured CPU results and the one-read feedback witness. The final verification section specifies exactly what Codex or a local operator must execute on the laptop.

From the extracted project root, the complete requested local workflow is:

```
python tools/verify.py --hardware --sanitizers --profile
```

A hosted workspace without the laptop GPU cannot complete this command's device obligations. The scripts do not install drivers, change privileged settings, alter clocks or power limits, or substitute CPU success for GPU verification. A blocked or failed stage is reported explicitly.

## 1.4 Integrity, reproducibility and privacy

SHA256SUMS.txt indexes the release's files, excluding itself. The verifier checks that manifest before building. assets/golden\_traces.json separately specifies the canonical CPU-derived output hashes. Extra local build/evidence directories are permitted, but changing an indexed release file fails integrity checking. The manifest detects changes relative to the distributed record; it is not a signature or independent proof of authorship.

All runs use fresh output folders. The binary writer refuses prior result artifacts rather than leaving an old success record attached to a failed run. Do not regenerate LUTs or golden files to make a mismatch disappear. Investigate the source of the mismatch and declare any intentional model change as a new revision.

This is a private-attribution edition. It includes the supplied identifier/date and the unmodified historical archive. Review those materials before public redistribution.

## 2 Explicit migration from Edition 3.0

Author: Tom Klootwijk · NL200678942 · 10-07-1990.

The requested basis is the delivered Edition 3.0 manuscript/kernel and its preserved source corpus. The log-encoded polar LUT/one-bit pipeline is also present on pages 3-6 of the double-arc addendum. Those pages are conceptual source material, not an implementation test. The present CUDA source, binary formats, integer coefficients and verification protocol are newly supplied engineering choices.

### 2.1 Retained foundations

The alphabet set remains  $\Sigma = \{X, Y, +, -, [, ]\}$ ; a finite word is a sequence, not a set that discards order or multiplicity. The complete seed is a structured machine input, not a solitary selector bit. The ordered XX and XY choices generate  $[X] [X]$  and  $[X] [Y]$ . Parallel productions are  $X \rightarrow X[+X]Y$  and  $Y \rightarrow Y[-Y]X$ ; turns and brackets retain themselves. At level g, the number of drawing symbols is  $2 \cdot 3^g$  and the word length is  $5 \cdot 3^g + 1$ .

The offline field uses the actual Edition 3.0 geometric interpreter and its default parameters: turtle step 0.12, angle  $\pi/7$ , shrink 0.72, translation (0,0.1), capsule radius 0.045, arc radius/spacing 1, and zero-memory arc thickness 0.105. Capsule/arc/baseline minimum composition and the final  $y \geq 0$  clipping are retained. `tools/geometry_v3.py` is a documented extracted subset of the earlier `kernel/core.py`. The complete original file remains in the original archive.

Binary latches, explicit threshold equalities, separate parity, integer pulse-density residuals, bounded memory, feedback and reproducible seeds are retained as ideas and formal interfaces. These are not licenses to equate geometry, chemistry and consciousness.

### 2.2 Deliberate changes - not an equivalence claim

| Edition 3.0                                                      | Native integer specialization 3.1                                                    |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Floating Cartesian graph masses and numerical transport substeps | Fixed log-polar graph with integer mass quanta and donor-limited integer transfers   |
| Global U fraction and three moving probes                        | A local post-reaction material fraction plus field observation at every lattice cell |
| Three global hysteresis latches                                  | One latch per lattice cell, packed in a separate bitplane                            |
| NumPy matrices and tanh recurrences                              | Explicit bounded integer weighted averages, with no floating reduction               |
| Incoming memory adjusts arc thickness                            | Incoming per-cell memory offsets a quantized sampled field                           |
| Dynamic analytical geometry evaluation                           | Five frozen level atlases, baked before execution and selected on device             |
| Conservative nearest-node remap when active mask changes         | Fixed storage lattice; closed/inactive cells retain their mass without moving it     |
| State-dependent analytical growth criterion                      | Global pulse count and dwell select the next pre-baked level                         |
| Five-bit payload plus parity at each global step                 | Four packed per-cell bitplanes; parity of each 32-pulse word is separately stored    |
| Continuous coordinates and analytical utilities                  | Finite native log-polar cell addresses; origin and infinity are not represented      |

The transport graph uses angular wrap and closed radial ends. Its rates are defined on graph adjacency, without polar metric/area factors. Therefore it is not a claim to discretize a physical polar Laplacian, calibrated diffusion, advection or chemistry. The two species are abstract, equally counted model mass units. The positive-total guard and exact divisors define their behavior.

No stochastic dither is used. Deterministic pulse-density encoding provides an exact dyadic count certificate; it does not promise blue-noise statistics, audio quality or lossless recovery of a continuous field. The 32-bit seed mixer is only a reproducible initializer and is not cryptography.

### 2.3 Source-to-code boundary

The corpus's terms "Word", "quintessence", spiritual interpretation, bodily analogy and consciousness remain documentary in the preserved Edition 3.0 corpus. They do not enter CUDA device selection, numerical coefficients or verification criteria. The author's name, identifier and supplied date are attribution metadata rather than secret numerical keys.

The archive is preserved as source history, not endorsed wholesale. Earlier unsupported or conflicting statements remain governed by the audits in Edition 3.0. This companion neither reinstates those assertions nor rewrites the user's corpus to make it appear to support a GPU result.

U3-D below is a theorem about the newly specified integer transition. Its relationship to the earlier model is structural and explicitly mapped above. No simulation-error theorem or exact behavioral quotient from the full Edition 3.0 state has been demonstrated. The exact CPU/GPU equivalence target is within U3-D, conditional on the CUDA implementation and hardware satisfying the stated execution contract.

### 3 Finite integer model and Unified Theorem U3-D

#### Tom Klootwijk Gambit - native CUDA engineering specialization 3.1

The theorem is about an explicitly bounded discrete machine. It is not an empirical theorem of consciousness. Written mathematical arguments, executable finite tests and prospective device measurements have different evidential roles.

#### 3.1 Complete finite seed

Let the complete seed be

$$\sigma_D = (\Sigma, \omega, P, \Theta, L, X_0, V, \mathcal{O}).$$

Here Sigma is the alphabet set, omega the ordered axiom, P the parallel grammar, Theta the validated integer/header and bake parameters, L the immutable packed LUT bytes, X0 the deterministic initial cell/control state, V the versioned implementation, and O the run options including pulse-feedback and any declared intervention. The selected step count belongs to O. All emitted file orderings are fixed. Documentary attribution fields do not change this mathematical machine.

The distributed chart has radial samples between  $r = 1/32$  and  $r = 4$  in dimensionless units. With R rows and A angular columns,

$$\rho_y = \log(1/32) + \frac{y}{R-1} \log(128), \quad \theta_x = 2\pi x/A, \quad p_{y,x} = e^{\rho_y} (\cos \theta_x, \sin \theta_x).$$

The radius origin is excluded. Only finitely many samples exist. A is a power of two between 32 and 2048; R is between 2 and 2048; levels G are in 1 through 5; and the run has 1 through 4096 updates. Consequently  $N = AR$  is divisible by 32, and no partial output word is required. Partially occupied thread blocks are nevertheless exercised by the verification asset.

#### 3.2 Offline field and packed LUT

At every admitted generation g, the Edition 3.0 zero-memory geometry produces a sampled field  $F_g$ . The packed code is

$$d_{g,y,x} = \text{clip}_{[-32768, 32767]} \left( \left\lfloor 4096 F_g(p_{y,x}) + \frac{1}{2} \right\rfloor \right),$$

$$c_{g,y,x} = \text{clip}_{[0, 65535]}(32768 - 16d_{g,y,x}), \quad L_{g,y,x} = (d_{g,y,x} + 32768) \mid (c_{g,y,x} \ll 16).$$

The low 16 bits use a biased signed-field representation; the upper 16 bits hold an unsigned field drive. The texture is unsigned 32-bit integer data, not normalized colour. Asset manifests record grammar words, sampling, saturation, hashes and the bake environment.

For an unsaturated *input sample value*, nearest-half-up quantization introduces at most  $1/8192$  field units of error. This statement is relative to the supplied floating sample. It is not an interval certificate for the error of the floating geometry evaluation itself. Away from a sample centre, finite sampling adds error. Saturation and field sign/threshold margins must be checked before drawing any continuous-space conclusion. The composite field is not asserted to be an exact interior distance to an overlapping union.

#### 3.3 State and computational self-reference

For cell i, let

$$X_i^n = (U_i^n, V_i^n, z_i^n, m_i^n, r_i^n, f_i^n, K_i^n, Q_i^n).$$

U and V are nonnegative integer mass counts; z and m are in  $[0, D]$ ; r is in  $[0, D-1]$ ;  $D = 65536$ . K and Q store cumulative codec input and pulse counts. Four flag bits store the latch, pulse, occupancy and latch event. The global control is (n,g,last-growth). State arrays use six 32-bit and two 64-bit fields per cell. That is 40 bytes, not one bit.

The previous pulse is decoded from the machine's **own packed output buffer**:

$$q_i^{n-1} = \left( W_{[i/32]}^{n-1} \gg (i \bmod 32) \right) \& 1.$$

The initial output buffer is zero. A declared one-bit intervention flips this read at one cell and one step only; it does not alter the historical output log. This creates a controlled counterfactual without corrupting the codec certificate.

With pulse-feedback enabled, the sampled angular bin is

$$j_i^n = \left( x_i + ns + q_i^{n-1}a \right) \bmod A,$$

where  $s = 1$  and  $a = 3$  in the supplied assets. The radial bin is unchanged. Incoming memory supplies

$$h_i^n = \text{trunc}_0((m_i^n - D/2)/256), \quad F_i^n = d_{g,y_i,j_i^n} - h_i^n,$$

$$a_i^n = \text{clip}_{[0,D]} \left( c_{g,y_i,j_i^n} + 16h_i^n + 8192(2q_i^{n-1} - 1) \right).$$

The divisors, angular shifts and gain are explicit header parameters. Signed division truncates toward zero, including for negative values. `--feedback 0` removes the pulse-dependent angular offset, memory field offset and direct pulse-drive term. It does not disable every other feedback loop: latch-mediated transport and pulse-count growth still operate. Therefore this option is described as **pulse-feedback ablation**, not a globally open-loop model.

Self-reference has this precise operational meaning: a descriptor produced by the machine becomes an input to its own subsequent transition. It does not mean a quine, a self-modifying executable, a model that proves all of its own properties, or subjective awareness.

### 3.4 Conservative transport and reaction

Each cell has up to four neighbors: angular predecessor/successor with wrap, and radial predecessor/successor where present. Radial ends have no exterior edge. If either sampled field on an edge is positive, its flux is zero. Otherwise the edge divisor is 8 when either incoming latch is one and 32 when both latches are zero. Both endpoints use the same edge divisor.

For one species M and each admitted directed edge i to j,

$$J_{i \rightarrow j} = \lfloor M_i / 2^{s_{ij}} \rfloor, \quad s_{ij} \geq 3,$$

$$\widetilde{M}_i = M_i - \sum_j J_{i \rightarrow j} + \sum_j J_{j \rightarrow i}.$$

Every output cell gathers from the *old* state; no endpoint writes into its neighbor. Closed/inactive cells retain their mass. No clipping hides negative counts and no new mass is invented at newly sampled geometry.

Next, the local reaction uses integer transfers

$$A_i = \lfloor \tilde{U}_i / 64 \rfloor, \quad B_i = \lfloor \tilde{V}_i / 128 \rfloor,$$

$$U'_i = \tilde{U}_i - A_i + B_i, \quad V'_i = \tilde{V}_i - B_i + A_i.$$

These are chosen graph-model transport and reaction laws, not calibrated molecular kinetics. Nonnegative integer coefficients make their conservation and positivity exact. The initial total mass must lie in 1 through  $2^{32}-1$ .

### 3.5 Sensing, bounded memory and the one-bit word

The post-reaction local material observation is

$$b_i = \begin{cases} \lfloor DU'_i / (U'_i + V'_i) \rfloor, & U'_i + V'_i > 0, \\ D/2, & U'_i + V'_i = 0. \end{cases}$$

Combine field and material, update fast state, then update memory:

$$v_i = \lfloor (3a_i^n + b_i)/4 \rfloor, \quad z'_i = \lfloor (3z_i^n + v_i)/4 \rfloor, \quad m'_i = \lfloor (15m_i^n + z'_i)/16 \rfloor.$$

The supplied implementation deliberately uses the new fast state in the memory update. The field sample for the current step still used incoming memory. Thus material, memory, bits and geometry participate in a causal loop without reading partially updated neighbors.

Set  $k_i = z_i$ . The codec is

$$q_i^n = \mathbf{1}\{r_i^n + k_i \geq D\}, \quad r'_i = r_i^n + k_i - Dq_i^n,$$

$$K'_i = K_i^n + k_i, \quad Q'_i = Q_i^n + q_i^n.$$

The latch uses threshold  $H = 49$  in field-code units:

$$\ell'_i = \begin{cases} 1, & F_i^n \leq -H, \\ 0, & F_i^n \geq H, \\ \ell_i^n, & -H < F_i^n < H. \end{cases}$$

The event is  $e_i = \text{old-latch XOR new-latch}$ ; raw occupancy is  $o_i = \mathbf{1}\{F_i \leq 0\}$ . Occupancy and a latch can legitimately disagree inside the hysteresis band. Those bits are not interchangeable.

Four planes pack pulses, latches, events and occupancies, each 32 cells per unsigned 32-bit word in least-significant-bit-first cell order. A fifth storage plane holds one 0/1 parity value per pulse word, in a 32-bit container. Parity is not mixed into the pulse-density count. It detects odd bit flips in a word, not all corruption or malicious changes.

After each step, the exact global pulse count  $P_n$  controls growth. If the dwell has elapsed, a level remains and  $4 P_n \geq N$ , increment the generation once and record the new last-growth step. This newly selected generation affects the following lookup. Supplied dwell is 16 steps. The finite pre-baked atlas ends at generation four.

### 3.6 Unified Theorem U3-D

**Statement.** Fix an admitted complete seed, valid run options, immutable LUT bytes, and the transition laws above. Assume fault-free execution of the specified integer operations, nonaliasing old/new buffers, ordered stage execution and the declared texture point-read semantics. Then through the admitted step budget:

1. The next state and ordered output are uniquely determined; replay is exact.
2. All represented masses remain nonnegative and their global sum is invariant.
3. Fast state and memory remain in  $[0,D]$ , residuals in  $[0,D-1]$ , and the stated integer widths suffice.
4. Every pulse and latch is binary. For every cell and every prefix  $t$ ,

$$D \sum_{n < t} q_i^n - \sum_{n < t} k_i^n = -r_i^t,$$

so the dyadic count discrepancy lies in  $(-1,0]$ .

5. Packing/unpacking is an exact encoding of each declared Boolean plane; events telescope in XOR and parity obeys its stated odd-error rule.
6. The described CUDA stage partition admits a schedule-independent realization of the same finite transition as the CPU reference. On conforming compiler/runtime/hardware execution, CPU and GPU therefore produce identical canonical states and words.
7. Computational self-feedback is a genuine causal input for the supplied witness. Removing or intervening on that input can change subsequent states; the reported one-bit output alone is not a sufficient full-state quotient.

Part 6 is an implementation-correctness target supported by the dependency argument below, not a claim that this delivery has executed or mechanically verified the CUDA compiler/hardware. Native device compilation, full equality tests and instrumentation remain explicit hardware obligations.

### 3.7 Supporting proof family

#### D1. Finite address and arithmetic closure

The validated dimensions give  $N \leq 2^{22}$  and at most  $5N$  LUT elements. The largest flattened LUT index fits unsigned 32 bits. Its  $y$  coordinate is less than 10240 and  $x$  less than 2048; adding one half to either integer is exactly representable in binary32. Hence texel-center address construction introduces no coordinate rounding for this admitted domain. The step/phase product is also within unsigned 32 bits.

The memory offset has magnitude at most 32768 even at the smallest admitted divisor. Field and drive intermediate expressions fit signed 64 bits (indeed their stated bounds fit signed 32 here). Recurrence numerators are at most  $16D$ . The codec sum is at most  $2D-1$ . The initial mixer alone intentionally wraps unsigned 32-bit products; wrap is not an accidental mass or codec operation.

#### D2. Integer transport positivity and conservation

Each donor has at most four neighbors and every divisor is at least eight. Therefore total outflow is at most  $4 \lfloor M_i/8 \rfloor \leq M_i/2 \leq M_i$ . Remaining mass and incoming terms are nonnegative. Sum the update over all cells: each directed  $J$  appears once negatively at its source and once positively at its destination. Every term cancels, so the species total is unchanged even when the realized gates differ at the next step.

The result is not a global contraction theorem for state-dependent geometry. It is a conservation/positivity theorem for each realized update.

### D3. Reaction positivity and conservation

$A \leq U$ -tilde and  $B \leq V$ -tilde. Subtracting the source's own transfer cannot make a species negative, and the reverse transfer is nonnegative. Adding the two updates cancels  $A$  and  $B$  exactly. Combining D2 and D3 proves invariant total represented mass. Every individual species count is at most the total, so the accepted initial bound ensures every stored count fits unsigned 32 bits. Intermediate gather sums and products use unsigned 64 bits.

### D4. Invariance of bounded observation and recurrent states

For nonzero total,  $0 \leq U'/(U'+V') \leq 1$ ; the chosen integer material observation is therefore in  $[0,D]$ . Its zero-total convention is also in that interval. A clipped field drive lies there by construction. Convex weighted integer averages, rounded downward, preserve  $[0,D]$ . Apply this first to  $v$ , then  $z'$ , then  $m'$ . Induction starts at  $z=m=D/2$ .

### D5. Exact prefix certificate

For  $0 \leq r < D$  and  $0 \leq k \leq D$ , the sum  $r+k$  lies in  $[0,2D-1]$ . Subtracting  $D$  exactly when this sum is at least  $D$  leaves a residual in  $[0,D-1]$ , with  $q$  in  $\{0,1\}$ . The identity  $Dq = k+r-r'$  telescopes over steps. Initial residual is zero, giving  $DQ-K = -r$  at every prefix. Dividing by  $D$  gives  $-1 < Q-K/D \leq 0$ . No floating-point error estimate is needed for this dyadic certificate.

There are at most 4096 steps, so  $K \leq 2^{28}$  per cell and global  $K \leq 2^{50}$ . The chosen unsigned 64-bit totals do not overflow. This is a certificate relative to integer  $k$ , not recovery of the original continuous geometry or the whole state.

### D6. Hysteresis and event identities

Positive  $H$  makes the two outer threshold conditions disjoint; the open middle band inherits a valid previous bit. Hence the next latch is binary, including at both specified equalities. Repeated XOR of events cancels every intermediate latch twice, leaving initial-latch XOR final-latch. Memory in the middle band is intentional rather than nondeterminism.

### D7. Bit packing and parity

Distinct cell offsets in a word use distinct powers of two. Extracting bit  $j$  after the sum/OR returns the original Boolean at position  $j$ . The word value stays within unsigned 32 bits, including bit 31. XOR reduction of all data bits defines parity. Flipping an odd number of data bits changes parity; an even number can preserve it. This is neither encryption nor authentication. The file SHA-256 manifest separately checks delivery integrity against an expected digest, not against an attacker who can replace both files and manifest.

### D8. Ordered stages and CPU/GPU equality

Preparation reads only incoming cells, incoming packed words, incoming control and immutable atlas data, then writes one sample per cell. Advancement reads those completed samples and old cells, then writes one new cell per thread. No cell writes into a neighbor. All launched lanes execute all four warp ballots; padded lanes contribute false. Each plane word has exactly one lane-zero writer, and only complete valid words are stored.

The fixed 256-thread summary uses synchronized shared-memory reductions. Addition is exact in the proven range, independent of grouping. It updates global control only after the cell transition

has completed. Sequential launches in one stream and explicit host copies order stages. The next iteration reads swapped buffers rather than in-place updates. Therefore every stage implements the scalar laws without schedule-dependent read/write choices. If the compiler/runtime/device honor these operations and texture semantics, induction gives CPU/GPU equality. Actual local execution must test this conditional conclusion; the written argument is not an nvcc or silicon proof.

#### **D9. Causal computational self-feedback witness**

The previous pulse is read from the engine's own packed result and appears in the next angular address and direct drive. For identical incoming states up to a selected read, flip that read at a cell whose clipping has not erased the effect. The resulting drive/address can differ, and thus the next fast state can differ. The supplied verification experiment at cell 7, step 3 observes this state difference while all earlier states match. Independent Python and C++ traces agree on that experiment. A separate pulse-feedback ablation changes the delivered bit trace.

This proves the specific operational feedback witness, not that every perturbation must change every trajectory or that a self-referential bitstream has experiences. Equal total numbers of ones can coexist with different timing and states.

#### **D10. Why the emitted bit is not the full machine**

Consider two admissible accumulator states that have just emitted  $q=0$  but retain residuals 0 and D-1. Feed both the same next  $k=1$ . Their next pulses are 0 and 1 respectively. Thus equality of the last emitted bit does not force equality of the next emitted bit under common input. The one-bit projection fails the exact quotient criterion retained from Edition 3.0. The complete finite seed, residual, memory, material, latches, generation and history cannot be silently discarded.

Combining D1-D10 establishes the stated mathematical conclusions and the conditional implementation relationship. Executable checks exercise finite cases. A future hardware pass adds device evidence; it does not turn the consciousness interpretation into an empirical result.

## 4 Native kernel implementation

### 4.1 A finite hardware target, not a placeholder shader

`src/gambit_cuda.cu` contains a native CUDA executable with checked allocations, texture creation, four named device kernels, full state/output checks and canonical binary writers. `include/gambit/model.hpp` contains the bounded integer transition functions compiled for both CPU and device. `src/host.cpp` supplies loading, validation, initialization, the scalar C++ reference and output serialization. It is not an empty framework adapter or an instruction to implement the model later. This is source delivery, however: it has not been compiled by `nvcc` in the delivery environment. A source-complete implementation is not the same as an executed GPU binary. Compiler and driver compatibility, code generation, device correctness and physical performance remain local verification obligations.

### 4.2 Read-only log-polar texture atlas

The five geometric generations are stacked vertically in one two-dimensional unsigned 32-bit CUDA array. A row is an angular strip at a fixed log radius. A generation has `height` consecutive rows. Array width is `width`; array height is `height * levels`. The field-code and drive channels are bit-packed into one texel.

`cudaCreateTextureObject` uses an array resource, point filtering, element-type reads and unnormalized coordinates. The fetch returns an unsigned integer, not a normalized float [H4]. The query integer is converted only to an exactly representable half-integer texel-centre address:

```
return tex2D<unsigned int>(tex,
    float(k % width) + 0.5f,
    float(k / width) + 0.5f);
```

Angular wrap is performed by integer masking before the fetch; clamp addressing is a secondary guard, not the mechanism implementing cyclic topology. Radial indexing remains in range. The texture is immutable for the entire run, avoiding same-kernel texture read/write coherence ambiguities [H4]. Dynamic memory deforms the sampled field code in registers; it does not overwrite cached texture storage.

The alternate `--fetch global` path indexes the same packed bytes through a global-memory pointer. The verifier compares both paths to the CPU. This is a correctness and profiling comparison, not an assumption that texture fetching will outperform a regular load. The GPU's cache behavior must be read from generated instructions and actual profiler evidence.

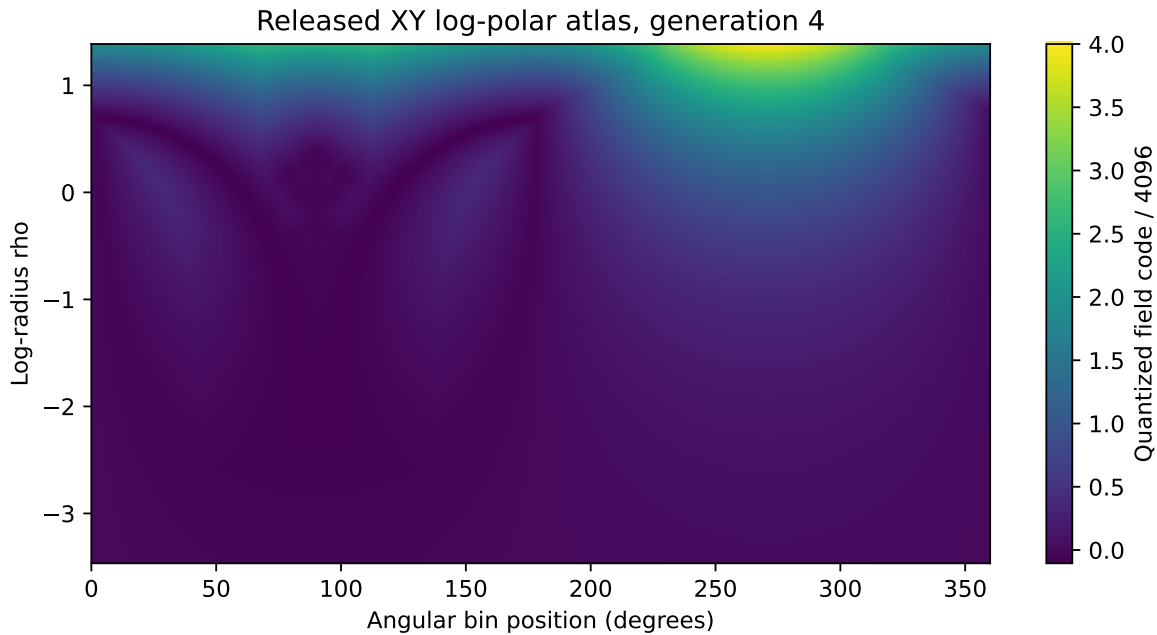


Figure 1: The actual distributed generation-four XY field in log-polar atlas coordinates. This is a finite sampled chart, not an infinite axis or a GPU performance measurement.

### 4.3 Ordered stages and unique writers

The update sequence has explicit stage boundaries:

```
incoming state + packed output + control
-> prepare: point lookup and feedback
-> advance: transport, reaction, state, words
-> summarize: integer counts and growth control
-> write the frame; swap old/new buffers
```

`gambit_prepare` fetches the immutable atlas and applies the incoming memory/pulse law. Each thread writes one sample. `gambit_advance` gathers transport from incoming cells and completed samples, applies reaction, sensing, memory and the one-bit codec, then writes one new cell. Four warp ballots pack the Boolean output planes. Every launched lane reaches every ballot; padding lanes contribute false. Lane zero writes each valid word exactly once.

`gambit_summarize` uses one fixed 256-thread block and a synchronized shared-memory integer reduction. Only its thread zero updates the global control. `gambit_texture_probe`, used before a verified run, fetches every atlas entry through the chosen texture/global path and compares the result against the original host bytes. Probe success establishes those reads for that run, not a universal cache guarantee.

Three update kernels are launched in order in the same stream. The host checks the summary, writes the emitted frame and swaps old/new state and word buffers. No mathematical state is updated in place. There are no floating-point reductions or atomic updates. The shared model header supplies identical integer operations, while the independent Python reference avoids relying only on shared source when validating the smaller cases.

## 4.4 Resource accounting for the supplied laptop profile

For  $N = 512 * 256$  cells and  $G = 5$  levels, the logical device payload is:

| Allocation                                      | Bytes             |
|-------------------------------------------------|-------------------|
| Two cell buffers, $2 * 40 N$                    | 10,485,760        |
| One sample buffer, $8 N$                        | 1,048,576         |
| Two sets of five packed/storage planes          | 163,840           |
| One unsigned-32-bit field atlas, $4 G N$        | 2,621,440         |
| Control and report                              | 76                |
| <b>Run payload</b>                              | <b>14,319,692</b> |
| Temporary full atlas probe in verification mode | 2,621,440         |
| <b>Conservative logical estimate with probe</b> | <b>16,941,132</b> |

The estimate is about 16.16 MiB with the probe. It excludes implementation-dependent texture layout, the CUDA context, runtime and driver allocations. The probe is actually freed before the persistent cell buffers are allocated, so the displayed combined estimate is deliberately conservative for these explicit buffers. Actual free memory is queried, allocation errors are checked, and the default seed cap is 2048 MiB. Reserving the entire advertised 12 GB is neither necessary nor desirable for this audit workload.

The history is written to files rather than retained as all timesteps in VRAM. One laptop-profile `words.bin` is 20,971,548 bytes, including all four packed Boolean planes plus the separate parity records and header. The final state file is 5,242,904 bytes. These sizes are part of the binary format, not a claim of lossless one-bit compression of the complete engine.

## 4.5 Execution boundaries

The device loop does not evaluate `log`, `sin`, `cos`, continuous SDF distances or L-system string rewriting. Those operations occur in the disclosed offline asset compiler. New dimensions, geometry or coefficients create a new complete seed and require new independently reviewed references. Regeneration of a shipped floating bake can change last-bit rounding and must not be used to hide a mismatch.

This native CUDA path still uses the operating system, NVIDIA driver/runtime, memory allocation, kernel launches, synchronization and host file I/O. It is not driverless firmware or an operating-system kernel. No display, audio actuator, transducer, network endpoint or biological interface is driven by this release. Outputs are model data files.

## 5 Canonical binary formats

All integers on disk are explicitly little-endian. No native C++ structure padding is serialized. CSV rows use LF newlines on both Windows and Linux. Runtime timing and hardware metadata are deliberately separate from the deterministic byte streams.

### 5.1 Packed LUT: `.gblut`

Header: 8 bytes `GBLUT31\0`; uint32 format version 1; uint32 parameter count 19; then nineteen uint32 values in the order below. Header size is exactly 92 bytes.

```
width, height, levels, start_level, steps, threshold,
feedback_gain, memory_divisor, phase_stride, feedback_angle,
open_shift, closed_shift, reaction_uv_shift, reaction_vu_shift,
growth_dwell, growth_divisor, seed, pair, device_budget_mib
```

The payload contains `levels * height * width` uint32 texels in generation-major, radial-major, angular-minor order. The low 16 bits are `signed_field_code + 32768`; the high 16 bits are the clamped field drive. `pair=0` selects XX and `pair=1` selects XY as symbolic configuration labels. It is not a chromosome data format.

The paired JSON manifest adds the SHA-256 digest, chart and bake metadata, complete grammar words and author attribution. Runtime arithmetic is determined by the actual validated header and texel bytes. The verifier compares them against the manifest, then against expected traces. The binary loader itself validates magic, header/version, safe parameter ranges and exact file length, but is not a cryptographic authenticator.

### 5.2 Packed one-bit words: `words.bin`

Header: 8 bytes `GBWORD31`, followed by five uint32 values: angular width, radial height, step count, storage-plane count ( $=5$ ), words per plane ( $=N/32$ ). Header size is 28 bytes.

Each timestep contains five consecutive arrays of  $N/32$  uint32 containers:

| Storage plane | Content          | Meaning of lane $j$ in container $w$                 |
|---------------|------------------|------------------------------------------------------|
| 0             | Pulse bits       | $q$ for cell $32*w+j$                                |
| 1             | Latch bits       | New persistent hinge state                           |
| 2             | Latch-event bits | Old hinge XOR new hinge                              |
| 3             | Occupancy bits   | Effective sampled field $\leq 0$                     |
| 4             | Parity records   | The whole uint32 is 0 or 1, parity of pulse word $w$ |

Planes 0-3 are genuinely bit packed. Plane 4 is a parity-record array, not 32 independent parity bits in each word. Parity is stored separately from pulses so it does not corrupt the pulse-density certificate. Latch or occupancy bits do not replace pulses in codec counts.

A textual rendering prints bits in increasing cell order: bit 0 first. This is deliberately the reverse of the usual most-significant-bit-first human display of an integer. `tools/show_words.py` implements the canonical cell order. File size is  $28 + \text{steps} * 20 * (N/32)$  bytes.

### 5.3 Complete final state: `state.bin`

Header: 8 bytes `GBSTAT31`, then uint32  $N$ . Every cell contributes six uint32 fields (`u`, `v`, `z`, `memory`, `residual`, `flags`) followed by two uint64 fields (`total_input`, `total_output`). Each cell is 40 bytes. Three uint32 control values (`step`, `level`, `last_growth`) follow all cells. Total file size is  $24 + 40*N$ .

Flag bit 0 is latch, bit 1 pulse, bit 2 occupancy and bit 3 latch event. All other flag bits are zero. All cells initially have zero flags, zero codec residual/counts, and  $z=\text{memory}=32768$ . Initial U and V are determined by the declared mixer and the initial level's quantized occupancy.

The final state supports per-cell `65536*total_output + residual == total_input`. `tools/check_trace.py` independently re-counts the emitted pulse bits per cell, checks latch-event telescoping and every parity record, compares final flags and verifies total mass/state consistency.

## 5.4 Summaries and expected hashes

`summary.csv` contains step, used/next generation, growth, counts, species totals, cumulative codec totals and error flags. `run.json` records backend, run options and nondeterministic operational metadata. `golden_traces.json` hashes only `words.bin`, `state.bin` and `summary.csv` for its disclosed cases.

A final-state certificate is not by itself an independent reconstruction of every prior coverage input. For that, execute the deterministic reference and compare states at every step, as the GPU verifier does, or use the independent Python specification on the disclosed small cases. A SHA-256 manifest proves equality with an expected file digest; it does not establish semantic correctness or authentication against replacement of both data and digest.

## 6 Executed evidence and binary self-feedback witness

### 6.1 Verification status at delivery

The delivery environment had GCC 14.2, Clang 17, CMake 3.31.6 and Python 3.13.5. It did not expose an NVIDIA device, `nvidia-smi` or `nvcc`. Consequently native CUDA compilation, GPU execution, texture-cache measurements and GPU sanitizer results are **NOT\_RUN / NOT\_MEASURED**. No GPU executable is included. The complete CUDA source and local build/verification workflow are provided for the laptop.

The C++ test executable evaluated **325,550 assertions** and passed. This is one CTest target with many evaluated assertions, not 325,550 independent test functions. The separate Python suite passed **19 test methods**, and the command-line/error-path suite passed **16 negative-path checks**. Release-mode checks use an explicit throwing check function rather than a disabled C++ `assert` macro.

Both GCC and Clang built the CPU implementation and passed its integer contract. Their XX/XY verification traces matched the same canonical hashes. A separate CPU AddressSanitizer/UndefinedBehaviorSanitizer build passed the synthetic contract test. These are CPU tool results, not NVIDIA Compute Sanitizer results.

Four small cases were recreated by both native C++ and the independent Python implementation. Their canonical `words.bin`, `state.bin` and `summary.csv` files matched byte-for-byte. Both laptop-sized native traces were recreated in fresh executions and matched their canonical hashes. Source changes or a new model require a declared revision; the laptop verifier does not overwrite the distributed references.

### 6.2 Actual CPU trace outcomes

| Case                       | Cells   | Steps | Pulse bits | Ones       | Conserved total mass |
|----------------------------|---------|-------|------------|------------|----------------------|
| XX verification            | 2,112   | 64    | 135,168    | 49,025     | 86,047               |
| XY verification            | 2,112   | 64    | 135,168    | 49,015     | 87,128               |
| XX laptop profile          | 131,072 | 256   | 33,554,432 | 12,169,202 | 5,208,103            |
| XY laptop profile          | 131,072 | 256   | 33,554,432 | 12,180,402 | 5,217,058            |
| XY pulse-feedback ablation | 2,112   | 64    | 135,168    | 50,864     | 87,128               |
| XY one-bit intervention    | 2,112   | 64    | 135,168    | 49,015     | 87,128               |

The two laptop-profile runs contain 67,108,864 pulse bits in total. This excludes the additional hinge, event, occupancy and parity storage. All six disclosed cases reached atlas level four, with the last growth advance at step count 48. The explicit integer trace audits passed. Counts are properties of these supplied discrete examples, not biological measurements.

### 6.3 The self-feedback counterfactual

The witness uses the released XY verification seed. Both runs have identical initial conditions and remain identical through updates 0, 1 and 2. At update 3, cell 7's read of its prior packed pulse is

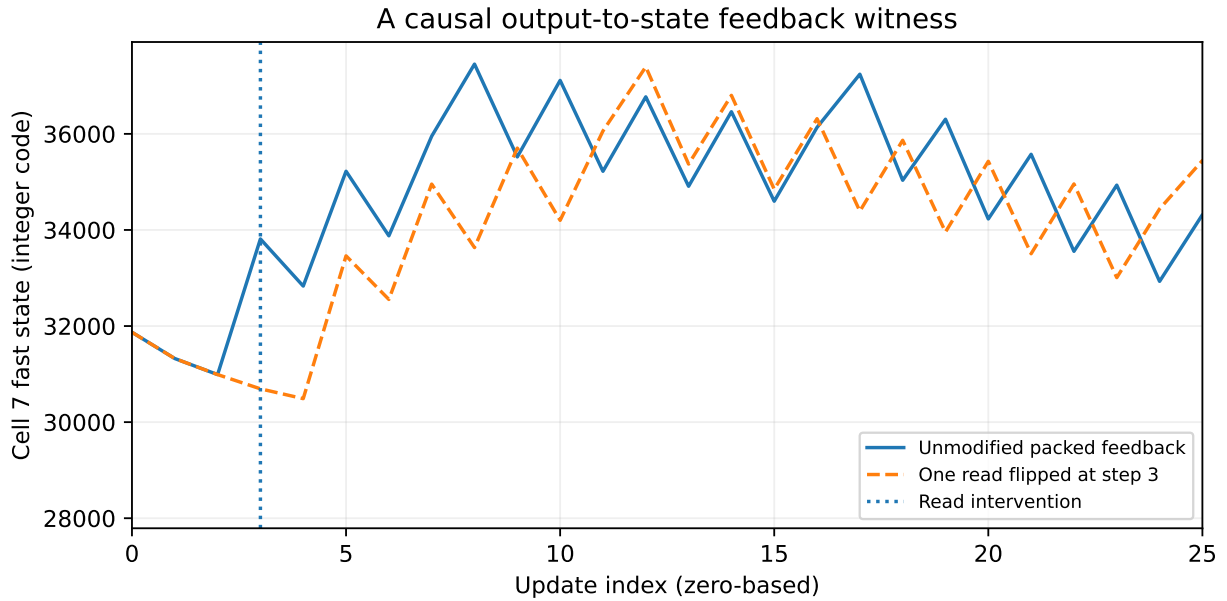


Figure 2: Independent Python witness: cell 7’s fast state differs after a single packed-feedback read is changed. The plotted update range is 0-25; the evidence file contains all 64 updates.

inverted once. Its stored previous bit was 1; the intervention supplies 0 to that read. No historical pulse is rewritten.

| Cell 7, after update 3 | Baseline | Read intervention |
|------------------------|----------|-------------------|
| Effective field code   | -122     | -106              |
| Prepared field drive   | 42,912   | 26,272            |
| Fast state z           | 33,813   | 30,693            |
| Memory m               | 32,601   | 32,406            |
| Codec residual r       | 62,462   | 59,342            |
| Material U / V         | 82 / 51  | 82 / 51           |
| Accumulated ones       | 1        | 1                 |

The first state divergence is therefore update 3. The first differing packed **pulse** bit occurs at update 7. Both complete 64-step runs happen to contain 49,015 ones: a total count can hide changes in timing and internal state. This distinction is why full words and states, not merely an aggregate checksum of counts, are compared.

The full witness is in `evidence/feedback_witness.json`; `tools/feedback_witness.py` independently reproduces it. The ablation also changes the output trace. These experiments demonstrate causal re-use of the engine’s output as an input. They are not a test of subjective awareness.

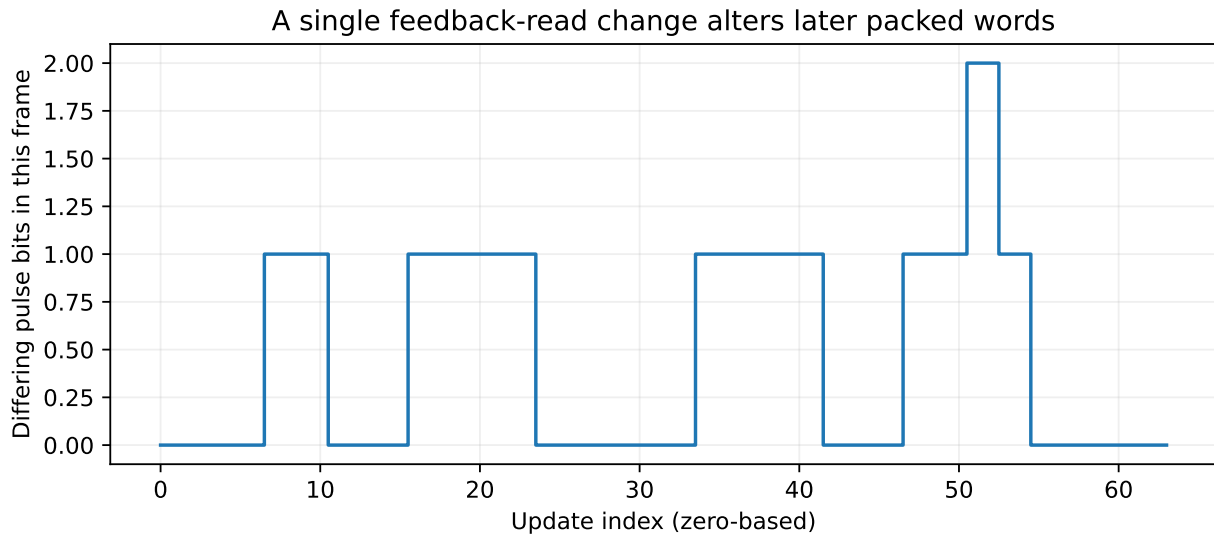


Figure 3: Frame-level pulse differences caused by the one-read intervention. Zero at an update does not imply equality of the hidden integer state.

#### 6.4 Words that can be inspected directly

`tools/show_words.py` prints a selected packed word in increasing cell-offset order: its leftmost printed character is bit 0, not bit 31. For the XY verification run, word 17 covers cells 544 through 575. The following are the first eight **pulse-plane** words, produced by the CPU reference:

```

step 0  00000000000000000000000000000000
step 1  00000000000000000000000000000000
step 2  11111111111111111111111111111111
step 3  00000000000000000000000000000000
step 4  11111111111111111111111111111111
step 5  000000000000000000000000000000001
step 6  11111111000000000011111111111110
step 7  00000000111111111100000000000011

```

The complete text excerpt additionally shows latch/event words and parity. These are data from an actual finite execution. The GPU is required to reproduce exactly these canonical words for the same seed and options, not simply generate visually similar patterns.

#### 6.5 What the evidence does not establish

The test suite is finite and the mathematical proofs depend on the stated model assumptions. No proof assistant has verified the C++ compiler, CUDA compiler or GPU. The original theory's physical, sensory and spiritual interpretation remains separate from this model. No result here establishes universal one-bit biology, a consciousness detector, infinite resolution, zero latency, or a measured texture-cache speedup.

## 7 Codex / local hardware verification contract

### 7.1 Required environment

The task must run in a shell with access to the user's **physical NVIDIA GeForce RTX 5070 Ti Laptop GPU**. A hosted workspace without that device is insufficient. NVIDIA lists the laptop model with 12 GB GDDR7; its device memory actually available to a process is measured at launch, not assumed from the label. The GeForce RTX 5070 Ti family is in compute-capability 12.0, and CUDA Toolkit 12.8.1's `nvcc` documentation explicitly lists `compute_120 / sm_120` [H1-H3].

Use CMake 3.24+, Python 3.10+ and a CUDA-supported C++17 host compiler. The package's native runtime does not require Python. The verifier does. CUDA still needs the NVIDIA driver/runtime [H4]. A number printed as "CUDA Version" by a driver utility is not used as proof that `nvcc` is installed: the script separately executes `nvcc --version` and `nvcc --list-gpu-code`.

On Windows, use the matching Visual Studio developer shell. On Linux, use a CUDA-supported host compiler and the native driver. The scripts do not install dependencies, contact an API, change privileged settings or upload data. Never silently fall back to emulation while reporting GPU success.

### 7.2 One local command

```
python tools/verify.py --hardware --sanitizers --profile
```

Open the extracted project directory as the local Codex workspace and ask it to follow **AGENTS.md**. OpenAI documents **AGENTS.md** as repository guidance read before work [H7]. The supplied file asks for verification before edits, preservation of evidence and precise reporting of blocked stages. It does not grant extra access to a GPU that the workspace cannot reach.

Suggested instruction to Codex:

Read AGENTS.md and verify this release on my RTX 5070 Ti Laptop GPU. Run the prescribed hardware command, preserve the original assets and goldens, and report the raw evidence path. Do not regenerate references to hide failures or claim consciousness from passing software tests.

### 7.3 What is run

The coordinator checks all files indexed in **SHA256SUMS.txt** before compiling. It builds/tests the CPU code, runs the independent Python tests and native negative-path tests, and recreates the supplied canonical goldens. Every verification-sized trace is compared byte-for-byte with a fresh Python scalar execution. Laptop-sized traces have native expected hashes and exact GPU/CPU state comparisons; the independent scalar Python run is intentionally restricted to the smaller cases.

The GPU stage checks actual device identity, compiler version and architecture target. It builds native `sm_120` code plus a `compute_120` PTX path. Each `--verify` GPU run first reads every LUT texel back through the selected fetch path. It then checks every prepared sample, every cell field, each packed plane, the complete control state and integer summary at every timestep against the C++ reference.

Both XX and XY verification assets run with block sizes 32, 128, 256 and 512. Their  $N = 2112$  cells leave padded thread blocks in the larger launches, while the bit words remain complete. A global-memory fetch backend is compared with the texture backend. Separate checks cover pulse-feedback ablation, a one-bit intervention, both 512-by-256 laptop assets, a fresh-process repeated run and the forced PTX-JIT route.

Compute Sanitizer is invoked with `memcheck`, `racecheck`, `initcheck` and `synccheck` on the disclosed eight-step XY verification case, with nonzero error exits [H5]. This is finite test coverage, not a

proof that every possible input has no defect. Racecheck principally checks shared-memory hazards; absence of its warnings cannot substitute for the old/new global-buffer dependency argument. The script saves PTX and SASS disassembly and collects Nsight Compute reports for both fetch modes [H6].

## 7.4 Cache evidence and performance interpretation

A texture object proves which API path was requested; generated instructions and profiler counters provide hardware evidence. Neither a source comment nor a repeated SHA-256 digest proves cache residency. Inspect the sampled kernels in the SASS and the texture/global memory workload reports. Record exact toolkit, driver, device, block size, profile, clock/power context and profiler configuration before comparing measurements.

The script requests `MemoryWorkloadAnalysis` and `LaunchStats` and saves the installed section listing. If an installed profiler version lacks these sections or hardware counters are unavailable, record the failure and adapt the *measurement command* explicitly. Do not disable security controls or change driver permissions without authorization. Captured reports still need interpretation; the verifier never asserts a speedup from their existence.

The emitted `wall_ms_including_io_and_oracle` includes host work, file writes and, in verification mode, CPU comparisons. It is **not kernel-only latency** and must not be used as a GPU speed benchmark. Profile the kernel regions separately. No cache-hit percentage, occupancy, register count, throughput, energy saving or nanosecond claim is supplied as a measured result in this release.

## 7.5 Finite resource policy

The default laptop seed uses 131,072 cells and five field levels. Two state buffers, one sample array, two sets of packed planes and one LUT are allocated; verification temporarily adds a full LUT readback array. The host streams frames to disk rather than accumulating the entire time history in VRAM.

The logical allocation estimate is checked against the asset's 2048 MiB cap and 60% of currently free device memory. Texture dimensions are checked against device limits. Actual array allocation/layout and driver overhead are implementation-dependent, and CUDA allocation failure still terminates the run. These guards are not a claim that all supported maximum-size jobs meet a laptop's watchdog or thermal budget. Do not turn off the watchdog, overclock or change power limits for this test.

## 7.6 Reports and failure meanings

Every invocation uses a new output folder. The native writer refuses pre-existing run artifacts, avoiding a stale success file after a failed run. `verification.json` records all command lines, exit codes, case results and instrumentation status. `run.json` distinguishes CPU passes, GPU invariant checks and GPU/CPU exact comparison passes.

Exit 0 means the specifically requested checks passed. Exit 1 means a check failed. Exit 2 means a required tool, device or access condition was blocked. `--cpu-only` can return 0 but reports `CPU_PASS_GPU_NOT_RUN`. `--quick` is explicitly limited and omits laptop-sized cases. A profiler block must not erase already collected functional evidence, nor be mistaken for a complete instrumentation pass.

No CI/cloud assertion, code inspection, CPU replay or symbolic theorem is a replacement for execution on the target device. A successful local run establishes that disclosed software/hardware test configuration. It does not prove consciousness or certify a biological or spiritual model.

## 8 Source and implementation references

Consulted 11 September 2026. Hardware/API references support engineering facts only; none establishes consciousness or validates the user’s interpretive claims.

### 8.1 Supplied project basis

[S1] Tom Klootwijk, *Gambit Unified Seed-Field Theorem and Theory of Consciousness - Edition 3.0*, supplied 35-page manuscript and accompanying archive. The original ZIP is preserved unchanged in `provenance/Tom_Klootwijk_Gambit_v3.0_original.zip`. It includes its source corpus, theorem U3, audits, Python kernel, configurations and historical evidence. Prior evidence is not presented as a CUDA test.

[S2] Supplied *double-arc-M-with-UU-double-set-SDF-notation-(signed-distance-field).pdf*, 12 pages; particularly pages 1-3 for arc/baseline notation and pages 3-6 for the conceptual log-polar LUT/one-bit pipeline. The earlier source’s numerical and physical claims remain subject to the Edition 3.0 audits. Page 6’s pseudocode is not an actual native CUDA implementation.

[S3] Supplied *Tom Klootwijk NL200678942 10-07-1990 Gambit.pdf*, 48 pages; specifically pages 13-16 (seed, grammar and field), 21-25 (transport/growth coupling), 38-42 (one-bit versus multiscale distinctions and assertions) and 47-48 (seed/alphabet notation). This is source history and project terminology, not independent scientific evidence.

### 8.2 Primary hardware and software documentation

[H1] NVIDIA, *GeForce RTX 50 Series Laptops*, specification table. The RTX 5070 Ti Laptop entry lists 12 GB GDDR7. <https://www.nvidia.com/en-us/geforce/laptops/50-series/>

[H2] NVIDIA Developer, *CUDA GPU Compute Capability*. The GeForce RTX 5070 Ti family appears under 12.0. Actual laptop identity/capability is queried locally. <https://developer.nvidia.com/cuda/gpus>

[H3] NVIDIA, *CUDA Compiler Driver NVCC*, CUDA 12.8.1 documentation, GPU architecture/code targets. Explicit `compute_120` and `sm_120` support. <https://docs.nvidia.com/cuda/archive/12.8.1/cuda-compiler-driver-nvcc/index.html>

[H4] NVIDIA, *CUDA C++ Programming Guide*, CUDA 12.8.1, texture/surface memory, texture object API, thread/warp execution and stream ordering. Element-type reads do not normalize values; integer point sampling is distinct from floating linear interpolation; same-kernel write/read texture coherence has restrictions. This package avoids modifying the texture during execution. <https://docs.nvidia.com/cuda/archive/12.8.1/cuda-c-programming-guide/index.html>

[H5] NVIDIA, *Compute Sanitizer User Guide*, tool roles and command-line usage. Racecheck is principally a shared-memory hazard detector. <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>

[H6] NVIDIA, *Nsight Compute CLI User Guide*, metrics, sections, export and profiler operation. Local measurements and their interpretation are still required. <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>

[H7] OpenAI, *Custom instructions with AGENTS.md*. Repository guidance for Codex; this does not supply unavailable local hardware. <https://developers.openai.com/codex/guides/agents-md/>

[H8] NVIDIA, *Blackwell Compatibility Guide*, CUDA 12.8.1, forced-PTX compatibility testing. The generic guide’s examples include datacenter targets; this package deliberately uses GeForce `sm_120`, not `sm_100`. <https://docs.nvidia.com/cuda/archive/12.8.1/blackwell-compatibility-guide/index.html>

The concrete integer model, formats, proof family, coefficients, source-to-code mapping, test cases and package layout are the engineering construction supplied in this release. Published API facts and vendor model specifications do not certify that construction without testing.